

# CSC490: Assignment 2

## Team Members

| Name            | Student ID |
|-----------------|------------|
| Aarya Prakash   | 1008790257 |
| Derek Huynh     | 1008849052 |
| Merrick Liu     | 1008951920 |
| Rhys Balevicius | 1000801974 |

University of Toronto  
Department of Computer Science

# 1 Aspirational Datasets

Our goal is to improve the efficacy of AI coding agents, which may produce incorrect changes because they lack relevant context that exists outside their immediate working scope. To build and evaluate systems that surface this missing context, the ideal dataset would consist of tuples pairing a coding task with a point-in-time snapshot of the full software environment: the target repository, its transitive dependencies, documentation, and version history along with a gold label identifying the precise, minimal set of non-local information an agent would have needed to produce a correct change. This label is the core artifact: it answers the question *"what did the agent need to know, but didn't, in order to get this right?"* and it spans multiple axes of context, including interface contracts and type signatures across module and library boundaries, downstream callers and invariants that a local change might violate, architectural conventions and design rationale recoverable from project history, and relevant documentation or specifications from external dependencies.

Each entry would additionally include a failed agent attempt (produced without sufficient context), the corresponding corrected code (produced by a human or by an agent given adequate context), and the failure signal linking the two such as a test failure, type error, or behavioural regression. The causal structure of this data is what makes it powerful: the delta between the failed and successful attempts is attributable to the presence or absence of the labelled context. With such a dataset, one could train and evaluate any retrieval or reasoning approach against a ground-truth definition of what "the right context" is for a given task, and measure whether surfacing it actually changes agent outcomes.

## Aspirational Dataset 1: Context-Attributed Agent Failures

Each row captures a coding task where an agent failed, the corrected outcome, and a precise attribution of what non-local context was missing.

| Task                |                                |                                                 |
|---------------------|--------------------------------|-------------------------------------------------|
| Field               | Type                           | Notes                                           |
| repo                | <code>str</code>               | Target repository                               |
| repo_snapshot       | <code>commit_sha</code>        | Frozen state of repo at task time               |
| dependency_snapshot | <code>[package@version]</code> | Pinned versions of all transitive dependencies  |
| task                | <code>str</code>               | Natural language description of the coding task |
| agent_attempt       | <code>AgentAttempt</code>      | Incorrect attempt record                        |
| failure_signal      | <code>FailureSignal</code>     | Failure manifestation                           |
| corrected_diff      | <code>Diff</code>              | Correct change                                  |
| missing_context     | <code>[MissingContext]</code>  | Required unseen context                         |

## Aspirational Dataset 2: Interface Contracts and Cross-Boundary Specifications

Each row describes a function or module boundary, its (possibly implicit) contract, and the set of callers/consumers that depend on that contract across repo and library boundaries.

| Function         |                      |                           |
|------------------|----------------------|---------------------------|
| Field            | Type                 | Notes                     |
| repo             | str                  | Repository                |
| repo_snapshot    | commit_sha           | Frozen state              |
| function         | fully_qualified_name | Canonical name            |
| defined_in       | {file, span}         | Definition location       |
| contract         | Contract             | Behavioural specification |
| consumers        | [Consumer]           | Call sites                |
| known_violations | [KnownViolation]     | Historical breaks         |

## Aspirational Dataset 3: Downstream Impact Chains

Each row captures a local change and the full causal chain through which it produces a failure at a distant location.

| ChangeImpact     |                        |                         |
|------------------|------------------------|-------------------------|
| Field            | Type                   | Notes                   |
| repo             | str                    | Repository              |
| repo_snapshot    | commit_sha             | Frozen state            |
| local_change     | Diff                   | Triggering modification |
| change_location  | {file, function}       | Origin of change        |
| failure_location | {file, function, test} | Where breakage surfaced |
| causal_chain     | [CausalNode]           | Propagation path        |
| chain_length     | int                    | Hop count               |
| minimal_context  | [{file, span}]         | Minimal foresight set   |

## Aspirational Dataset 4: Architectural Conventions and Project Norms

Each row encodes an implicit or explicit convention in a codebase, examples of code that follow it, and examples of agent-generated code that violates it.

### ConventionSchema

| Field               | Type           | Notes             |
|---------------------|----------------|-------------------|
| repo                | str            | Repository        |
| repo_snapshot       | commit_sha     | Frozen state      |
| convention          | Convention     | Normative pattern |
| conforming_examples | [{file, span}] | Correct usages    |
| violations          | [Violation]    | Broken instances  |

## Aspirational Dataset 5: Design Rationale and Temporal Context

Each row links a region of code to the historical decisions that explain *why* it is the way it is: the context an agent needs to avoid "cleaning up" intentional complexity.

### CodeHistory

| Field           | Type            | Notes                          |
|-----------------|-----------------|--------------------------------|
| repo            | str             | Repository                     |
| repo_snapshot   | commit_sha      | Frozen state                   |
| code_region     | {file, span}    | Location                       |
| current_form    | str             | Present code                   |
| history         | [HistoryEntry]  | Prior commits                  |
| rationale       | Rationale       | Design justification           |
| naive_refactors | [NaiveRefactor] | Tempting but incorrect changes |

## Aspirational Dataset 6: Cross-Repository Dependency Context

Each row captures a usage of an external library and the external documentation, source, or changelog context needed to use it correctly.

### DependencyUsage

| Field               | Type                 | Notes               |
|---------------------|----------------------|---------------------|
| consumer_repo       | str                  | Consuming repo      |
| repo_snapshot       | commit_sha           | Frozen state        |
| dependency          | {name, version}      | External package    |
| usage_location      | {file, span}         | Call site           |
| api_used            | fully_qualified_name | Referenced symbol   |
| required_context    | [RequiredContext]    | External knowledge  |
| version_sensitivity | VersionSensitivity   | Version constraints |
| misuse_examples     | [MisuseExample]      | Incorrect usages    |

- 2 Reality Check
- 3 Data-processing pipelines
- 4 IaC Implementation
- 5 Disaster Recovery Demonstration